CSE 344 – System Programming
Homework #6
Magic Academy
Due Date: 20.05.2026 – 23:59

Important Rules

• Implemented in C, must compile and run on Debian 12 (64-bit) in VirtualBox.

• Identical or suspiciously similar submissions receive -100 points and are reported for academic misconduct.

• A report with uncut screenshots is mandatory. A submission without a report is not evaluated.

• Late submissions are NOT accepted.

• Post all questions in the Teams assignment thread so every student benefits from the answers.

1. Overview

In this homework you will design and implement a magic academy server and two client programs. The server handles multiple simultaneous TCP clients using select() or poll() for I/O multiplexing (no thread-per-client). The server enforces a maximum number of simultaneous clients and automatically disconnects idle clients after a configurable timeout period.

This homework covers:

- TCP sockets for all client-server communication
- All client connections managed in a single loop without threads
- Per-client state management: each connected TCP client has a username, type, and spellbook
- Dynamic ingredient updates: BREW and CONSUME orders change ingredient quantities
- Idle client timeout: clients inactive for more than TIMEOUT seconds are automatically disconnected
- Maximum client enforcement: server rejects new connections when capacity is full
- Partial read/write handling: TCP does not guarantee message boundaries
- Signal handling: server and clients exit cleanly on SIGINT (Ctrl+C).

⚠ The server must be single-threaded. All TCP I/O must be handled inside one select() or poll() loop. Thread-per-client is strictly forbidden and will result in a major penalty.

2. System Scenario

Hogwarts opens its gates every morning. The server loads a list of magical ingredients and their initial quantities from a file. Two types of participants connect to the academy:

- **Wizard**: connects via TCP, submits BREW and CONSUME orders, checks their spellbook inventory
- **Professor:** connects via TCP, queries ingredient quantities and connected client information.

When a wizard submits a BREW or CONSUME order, the server updates the ingredient quantity and logs the transaction. Professors can query current quantities at any time. Each wizard has a personal spellbook stored on the server: the server tracks how many units of each ingredient the wizard holds.

The server enforces two additional constraints:

- **Maximum capacity**: Hogwarts accepts at most MAX_CLIENTS simultaneous TCP connections. When full, any new connection is immediately sent ERR HOGWARTS_FULL and closed.
- **Idle timeout**: any client that sends no command for TIMEOUT seconds is automatically disconnected with the message TIMEOUT DISCONNECT sent to the client, and the event is logged.

On SIGINT: the server sends a disconnect notice to all TCP clients, closes all file descriptors, and exits cleanly.

3. Programs to Implement

Three separate programs must be implemented and compiled:

Program	Protocol	Role
hogwarts	TCP	Academy server. Manages all clients, ingredients, spellbooks..
wizard	TCP	Submits BREW and CONSUME orders. Maintains a spellbook on the server.
professor	TCP	Queries ingredient quantities.

3.1 Command-Line Parameters

Flag	Parameter	Description
-p	<tcp_port>	TCP port for wizard and professor connections. Must be >= 1024.
-s	<ingredients.txt>	Path to ingredient list file. Must exist and be readable.
-l	<logfile>	Path to server log output file.
-n	<max_clients>	Maximum number of simultaneous TCP clients. Must be >= 1.
-t	<timeout>	Idle timeout in seconds. Must be >= 1.

Missing or invalid parameters must print a usage message to stderr and exit with a non-zero code.

3.2 hogwarts

./hogwarts -p <tcp_port> -s <ingredients.txt> -l <logfile> -n <max_clients> -t <timeout>

- Loads ingredient list and initial quantities from ingredients.txt at startup
- Opens a TCP socket on tcp_port (accepts wizard and professor connections)
- Uses select() or poll() to multiplex all TCP client fds in a single thread
- Maintains per-client state: username, type (WIZARD/PROFESSOR), fd, line buffer, spellbook (WIZARD only), last activity timestamp
- Enforces maximum capacity: rejects new connections with ERR HOGWARTS_FULL when max_clients is reached
- Enforces idle timeout: disconnects any client that has not sent a command in the last TIMEOUT seconds with TIMEOUT DISCONNECT
- Recalculates the select() timeout before every call to enforce idle checking without threads or sleep()
- Supports at least max_clients simultaneous TCP clients as specified by -n
- On SIGINT: sends SERVER SHUTDOWN to all TCP clients, closes all fds, exits cleanly

3.3 wizard

./wizard <server_ip> <tcp_port> <username>

- Connects to server via TCP
- Automatically sends ENROLL WIZARD <username> upon connection
- Reads commands from stdin, sends to server
- Prints all server responses to stdout
- When the user presses Ctrl+C, the client sends APPARATE and exits cleanly

3.4 professor

./professor <server_ip> <tcp_port> <username>

- Connects to server via TCP
- Automatically sends ENROLL PROFESSOR <username> upon connection
- Can only send: INSPECT, SCROLL, ROSTER, APPARATE
- Cannot send BREW or CONSUME: server rejects these with ERR UNAUTHORIZED
- When the user presses Ctrl+C, the client sends APPARATE and exits cleanly

4. Input Files

4.1 ingredients.txt

Each line defines one magical ingredient:

<ingredient> <initial_quantity>

Example:

MOONSTONE 100
DRAGON_SCALE 50
PHOENIX_FEATHER 30
MANDRAKE_ROOT 75
UNICORN_HAIR 20

- ingredient: uppercase alphanumeric with underscores, max 16 characters
- initial_quantity: positive integer
- Lines with invalid format must be skipped silently

5. Protocol

All TCP messages are line-based: each message ends with \n. Maximum line length: 512 bytes. Partial reads must be handled with a per-client line buffer. Partial writes must also be handled correctly when sending responses.

5.1 Wizard Commands (TCP)

Command	Description	Server Response
ENROLL WIZARD <username>	Register as wizard. Must be first command.	OK ENROLL <username> or ERR ENROLL name_taken
BREW <ingredient> <qty>	adds ingredient to both global stock and wizard spellbook.	OK BREW <ingredient> <qty> <new_total> or ERR UNKNOWN_INGREDIENT
CONSUME <ingredient> <qty>	removes ingredient from both global stock and wizard spellbook.	OK CONSUME <ingredient> <qty> <new_total> or ERR INSUFFICIENT_INGREDIENTS
SPELLBOOK	Request own inventory.	OK SPELLBOOK <ingredient>:<qty>,... or OK SPELLBOOK EMPTY
APPARATE	Disconnect.	OK APPARATE

5.2 Professor Commands (TCP)

Command	Description	Server Response
ENROLL PROFESSOR <username>	Register as professor. Must be first command.	OK ENROLL <username> or ERR ENROLL name_taken
INSPECT <ingredient>	Query current global quantity of ingredient.	OK INSPECT <ingredient> <qty> or ERR UNKNOWN_INGREDIENT
SCROLL	Get full ingredient snapshot.	OK SCROLL <ingredient>:<qty>,...
ROSTER	List all connected client usernames.	OK ROSTER <u1>,<u2>,...
APPARATE	Disconnect.	OK APPARATE

5.3 Connection Rejection (Server to New Client)

When the server has reached max_clients capacity, any new incoming connection is handled as follows:

- Server accepts the connection with accept()
- Immediately sends ERR HOGWARTS_FULL
- Closes the file descriptor
- Logs the rejection

This ensures the client receives a meaningful error instead of a silent timeout.

5.4 Idle Timeout Mechanism

The server tracks the last activity timestamp for each connected client using time() or clock_gettime(). Before each select() call, the server:

- Iterates over all connected clients
- Calculates time elapsed since last activity for each client
- Disconnects any client whose elapsed time exceeds TIMEOUT seconds by sending TIMEOUT DISCONNECT and closing the fd
- Recalculates the minimum remaining time among all clients
- Passes this minimum as the timeout parameter to select()

⚠ The idle timeout must be implemented without sleep() or threads. The remaining time must be recalculated before every select() call using clock_gettime() or time().

5.5 Ingredient Update Mechanism

Ingredient global quantities change dynamically based on wizard orders:

- BREW <ingredient> <qty>: global quantity increases by qty
- CONSUME <ingredient> <qty>: global quantity decreases by qty
- Global quantity must never go below 0
- Quantity is stored and reported as a non-negative integer

5.6 Per-Client State

The server maintains a state record for each connected TCP client:

- username: registered name, from ENROLL command
- type: WIZARD or PROFESSOR
- fd: file descriptor
- line_buf: partial read buffer, handles incomplete TCP messages
- last_active: timestamp of last received command, used for timeout enforcement
- spellbook: WIZARD only, ingredient to quantity mapping

The server must maintain a client_t struct for each connected client. A minimum required structure is:

```
typedef struct {
    int      fd;
    char     username[32];
    char     type[16];       /* WIZARD or PROFESSOR */
    char     line_buf[512];  /* partial read buffer */
    time_t   last_active;    /* timestamp of last command */
    /* spellbook: design is left to you (WIZARD only) */
} client_t;
```

When a client disconnects (APPARATE, timeout, or hangup), its state must be freed and its fd removed from the fd_set. Spellbook data is not persisted: it is lost on disconnect.

Common error responses for both client types:

- ERR ENROLL name_taken: username is already in use. Two clients cannot share the same username regardless of type
- ERR UNAUTHORIZED: command not allowed for this client type
- ERR UNKNOWN <command>: unrecognized command
- ERR TOOLONG: line exceeds 512 bytes
- ERR NOT_ENROLLED: client sent a command before ENROLL
- ERR HOGWARTS_FULL: server is at maximum capacity
- TIMEOUT DISCONNECT: client was idle too long, server is closing the connection
- SERVER SHUTDOWN: server is closing, client must disconnect

6. Outputs

6.1 Output Keywords

All output lines follow the format: [SOURCE] KEYWORD fields. The table below covers both server and client output lines.

Keyword	Source	Required Fields
SERVER_STARTED	[SERVER]	port=N max_clients=M timeout=T ingredients=K
CLIENT_CONNECTED	[SERVER]	fd=N ip=X.X.X.X
ENROLL	[SERVER]	username=X type=WIZARD/PROFESSOR fd=N
BREW	[SERVER]	wizard=X ingredient=Y qty=N old_qty=P new_qty=Q
CONSUME	[SERVER]	wizard=X ingredient=Y qty=N old_qty=P new_qty=Q
INSPECT	[SERVER]	professor=X ingredient=Y qty=N
SCROLL	[SERVER]	professor=X ingredients=K
ROSTER	[SERVER]	professor=X clients=N
REJECTED	[SERVER]	fd=N ip=X.X.X.X reason=HOGWARTS_FULL
TIMEOUT	[SERVER]	username=X fd=N elapsed=Ns
CLIENT_DISCONNECTED	[SERVER]	username=X reason=APPARATE/hangup/timeout
SHUTDOWN	[SERVER]	signal=SIGINT
CLEANUP_DONE	[SERVER]	clients=0
CONNECTED	[WIZARD/PROFESSOR username]	server=IP:port
DISCONNECTED	[WIZARD/PROFESSOR username]	reason=APPARATE/shutdown/timeout
SENT	[WIZARD/PROFESSOR username]	command text
RECEIVED	[WIZARD/PROFESSOR username]	server response text

⚠ **Output line prefix format is mandatory.** [SERVER], [WIZARD username], [PROFESSOR username]. Wrong prefix or missing fields will fail grading checks.

6.2 Example Server Log (stdout + server.log)

All server events must be written to the logfile specified by -l. Each line is timestamped. The following events must be logged.

The server logfile:

```
[2026-05-08 14:32:01] SERVER_STARTED port=5000 max_clients=8 timeout=30 ingredients=5
[2026-05-08 14:32:10] CLIENT_CONNECTED fd=5 ip=127.0.0.1
[2026-05-08 14:32:10] ENROLL username=hermione type=WIZARD fd=5
[2026-05-08 14:32:15] CLIENT_CONNECTED fd=6 ip=127.0.0.1
```

```
[2026-05-08 14:32:15] ENROLL username=dumbledore type=PROFESSOR fd=6
[2026-05-08 14:32:20] BREW wizard=hermione ingredient=MOONSTONE qty=10 old_qty=100 new_qty=110
[2026-05-08 14:32:25] CONSUME wizard=hermione ingredient=DRAGON_SCALE qty=5 old_qty=50 new_qty=45
[2026-05-08 14:32:30] INSPECT professor=dumbledore ingredient=MOONSTONE qty=110
[2026-05-08 14:32:35] SCROLL professor=dumbledore
[2026-05-08 14:32:40] ROSTER professor=dumbledore
[2026-05-08 14:32:45] TIMEOUT username=hermione fd=5 elapsed=30s
[2026-05-08 14:32:45] CLIENT_DISCONNECTED username=hermione reason=timeout
[2026-05-08 14:32:55] REJECTED fd=7 ip=127.0.0.1 reason=HOGWARTS_FULL
[2026-05-08 14:32:58] CLIENT_DISCONNECTED username=dumbledore reason=APPARATE
[2026-05-08 14:33:00] SHUTDOWN signal=SIGINT
[2026-05-08 14:33:00] CLEANUP_DONE clients=0
```

The server prints a startup confirmation to stdout:

```
Hogwarts is ready. Port: 5000 | Max Clients: 8 | Timeout: 30s
```

6.3 Example Client Output (stdout only)

All server responses are printed to stdout exactly as received. Additionally:

Example wizard session:

```
$ ./wizard 127.0.0.1 5000 hermione
CONNECTED server=127.0.0.1:5000
RECEIVED OK ENROLL hermione
> BREW MOONSTONE 10
SENT BREW MOONSTONE 10
RECEIVED OK BREW MOONSTONE 10 110
> SPELLBOOK
SENT SPELLBOOK
RECEIVED OK SPELLBOOK MOONSTONE:10
> APPARATE
SENT APPARATE
RECEIVED OK APPARATE
DISCONNECTED reason=APPARATE
```

Example professor session:

```
$ ./professor 127.0.0.1 5000 dumbledore
CONNECTED server=127.0.0.1:5000
RECEIVED OK ENROLL dumbledore
> INSPECT MOONSTONE
SENT INSPECT MOONSTONE
RECEIVED OK INSPECT MOONSTONE 110
> SCROLL
SENT SCROLL
RECEIVED OK SCROLL MOONSTONE:110,DRAGON_SCALE:45,PHOENIX_FEATHER:30
```

```
> ROSTER
SENT ROSTER
RECEIVED OK ROSTER hermione,dumbledore
> APPARATE
SENT APPARATE
RECEIVED OK APPARATE
DISCONNECTED reason=APPARATE
```

7. Signal Handling and Shutdown

7.1 Server Shutdown (Ctrl+C)

On SIGINT (Ctrl+C) sent to the server:

- The signal handler sets a volatile sig_atomic_t flag using only async-signal-safe operations.
- The main select() loop detects the flag and enters shutdown mode.
- SERVER SHUTDOWN is sent to all connected TCP clients.
- All TCP file descriptors are closed.
- SHUTDOWN and CLEANUP_DONE are logged.
- No zombie processes or orphan file descriptors may remain after exit.

7.2 Client Exit (Ctrl+C)

When the user presses Ctrl+C (SIGINT) on a wizard or professor terminal:

- The signal handler sets a volatile sig_atomic_t flag using only async-signal-safe operations. The main loop detects the flag and initiates exit.
- The client sends APPARATE to the server before closing.
- The server responds with OK APPARATE and logs CLIENT_DISCONNECTED reason=APPARATE.
- The client closes the TCP connection and exits cleanly.

7.3 Idle Timeout Disconnect

When the server detects an idle client (no command received within TIMEOUT seconds):

- The server sends TIMEOUT DISCONNECT to the client.
- The client prints a message and exits cleanly.
- The server logs TIMEOUT username=X fd=N elapsed=Ns and CLIENT_DISCONNECTED reason=timeout.
- The server removes the client from the fd_set and frees its state.

7.4 Server Full Rejection

When a new client attempts to connect but max_clients is already reached:

- The server accepts the connection with accept().

- Immediately sends ERR HOGWARTS_FULL.
- Logs REJECTED fd=N ip=X.X.X.X reason=HOGWARTS_FULL.
- Closes the file descriptor without adding it to the client list.

8. Submission

Compress as StudentID_HW6.zip. Do not include binaries or .o files.

```
<StudentID>_hw6/
├── hogwarts.c
├── wizard.c
├── professor.c
├── (and any additional .c / .h files)
├── Makefile
└── StudentID_report.pdf
```

Makefile must compile all three programs with gcc -Wall -Wextra -std=c11 and include make clean.

Example:

```
make       # builds all
make clean  # removes all binaries and .o files
```

9. Mandatory Test Scenarios

Run all scenarios and include uncut terminal screenshots in your report. Each scenario requires multiple terminals open simultaneously. Screenshot each terminal separately. Start with a fresh server for each scenario unless stated otherwise.

Scenario 1

- ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 30
- ./wizard 127.0.0.1 5000 hermione
- ./professor 127.0.0.1 5000 dumbledore
- ./wizard 127.0.0.1 5000 hermione *(same username)*
- Connect a third client, send any command before ENROLL
- From hermione: FLY BROOMSTICK
- From dumbledore: BREW MOONSTONE 5
- From dumbledore: CONSUME MOONSTONE 5
- From hermione: INSPECT MOONSTONE
- From hermione: SCROLL
- From hermione: Ctrl+C
- From dumbledore: Ctrl+C

Scenario 2

- ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 4 -t 30
- ./wizard 127.0.0.1 5000 harry
- ./professor 127.0.0.1 5000 mcgonagall
- From harry: BREW MOONSTONE 10
- From harry: SPELLBOOK
- From harry: BREW UNICORN_HAIR 15
- From harry: SPELLBOOK
- From harry: CONSUME MOONSTONE 5
- From harry: CONSUME UNICORN_HAIR 20
- From harry: BREW BUTTERBEER 5
- From mcgonagall: INSPECT DRAGON_SCALE
- From mcgonagall: SCROLL
- From mcgonagall: ROSTER
- From mcgonagall: INSPECT BUTTERBEER
- From harry: Ctrl+C
- From mcgonagall: Ctrl+C

Scenario 3

- ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 3 -t 60
- ./wizard 127.0.0.1 5000 harry
- ./wizard 127.0.0.1 5000 ron
- ./professor 127.0.0.1 5000 dumbledore
- ./wizard 127.0.0.1 5000 voldemort
- ./professor 127.0.0.1 5000 snape
- From harry: Ctrl+C
- ./wizard 127.0.0.1 5000 neville
- From neville: BREW MOONSTONE 5
- From dumbledore: ROSTER
- From ron: Ctrl+C
- From dumbledore: Ctrl+C
- From neville: Ctrl+C

Scenario 4

- ./hogwarts -p 5000 -s ingredients.txt -l server.log -n 6 -t 10
- ./wizard 127.0.0.1 5000 hermione
- ./wizard 127.0.0.1 5000 harry
- From hermione: BREW MOONSTONE 5
- (wait 3 seconds)
- From hermione: SPELLBOOK
- (wait 3 seconds)
- From hermione: BREW DRAGON_SCALE 2
- (harry sends nothing — wait 10 seconds)
- From hermione: SPELLBOOK
- ./wizard 127.0.0.1 5000 ron
- ./professor 127.0.0.1 5000 dumbledore
- ./professor 127.0.0.1 5000 mcgonagall
- From ron: BREW UNICORN_HAIR 5
- From dumbledore: SCROLL
- From mcgonagall: ROSTER
- From server terminal: Ctrl+C
- ps aux | grep hogwarts
- cat server.log

⚠ All screenshots must be uncut, showing the full terminal from command prompt through program exit. Cropped or edited screenshots will not be accepted.

10. Report Requirements

The report must explain design decisions, not just describe the code. Diagrams are required.

Section	Content
Title Page	Name, ID, course, HW number
System Design	How the server manages all TCP clients in a single loop without threads. How idle timeout is recalculated before every iteration. How new connections are rejected when the server is at full capacity.
Data Structures	Structure of client_t. How spellbook is managed per wizard. How ingredient list is stored.
Partial Read/Write Handling	How line buffers handle incomplete TCP messages.
Signal Handling and Shutdown	Step-by-step server Ctrl+C sequence. Wizard and professor Ctrl+C sequence.

Test Scenarios	Uncut screenshots for all 4 mandatory tests with explanation.
Challenges	At least 2 concrete problems encountered and how they were resolved.

11. Grading and Penalties

Grading is holistic. A missing or non-functional component affects the correctness of all criteria that depend on it. If select() is not used, multi-client handling cannot be graded. If per-client state is absent, spellbook tracking and authorization enforcement cannot be verified.

Grading

Criterion	Points
TCP server: select/poll loop, multi-client, correct lifecycle	15
Protocol correctness: all commands, correct responses, ERR cases	10
Per-client state: spellbook tracking, type enforcement, cleanup on disconnect	10
Ingredient update mechanism: BREW/CONSUME updates global and per-wizard quantities correctly	5
Capacity enforcement: HOGWARTS_FULL after accept(), slot recovery on disconnect	5
Idle timeout: recalculated before every select(), TIMEOUT DISCONNECT sent correctly	10
Signal handling: server SIGINT clean shutdown, wizard/professor SIGINT (Ctrl+C) graceful exit	10
Output format correctness (server + clients)	10
Compilation, Makefile (all 3 programs), zero -Wall -Wextra warnings	5
Report: all sections, 4 test screenshots uncut	20
TOTAL	100

Penalties

Condition	Penalty
Code does not compile	-100
No report submitted	Not evaluated
No Makefile	Not evaluated
Late submission	Not accepted
Report submitted but screenshots missing or cropped	-80

Missing or broken Makefile	-80
Thread-per-client used instead of select/poll	-60
Server handles only one TCP client at a time	-60
Idle timeout implemented with sleep() or a thread	-40
Per-client state absent, no spellbook tracking	-40
Professor can send BREW/CONSUME without rejection	-30
Partial read/write not handled	-30
HOGWARTS_FULL not enforced	-30
Duplicate username accepted	-25
Crash on invalid or unknown command	-25
File descriptor leak after client disconnect	-25
Each -Wall warning (first 10)	-2 each
More than 10 -Wall warnings	-25